

TUGAS PRAKTIKUM 6

Deadlock



Dian Chandra M

2410651032

**UNIVERITAS MUHAMMDIYAH JEMBER
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INFORMATIKA**

2025

Tugas Praktikum

Tugas 1: Analisis Deadlock

Jalankan program `deadlock_simple.c` dan jelaskan:

```
chandra@linux@LAPTOP-E5CDVDCL:~$ nano deadlock_simple.c
chandra@linux@LAPTOP-E5CDVDCL:~$ gcc deadlock_simple.c -o deadlock_simple -lpthread
chandra@linux@LAPTOP-E5CDVDCL:~$ ./deadlock_simple
=== SIMULASI DEADLOCK ===

Thread 1: Mencoba mengambil Lock 1...
Thread 1: Berhasil dapat Lock 1!
Thread 2: Mencoba mengambil Lock 2...
Thread 2: Berhasil dapat Lock 2!
Thread 1: Mencoba mengambil Lock 2...
Thread 2: Mencoba mengambil Lock 1...
^C
chandra@linux@LAPTOP-E5CDVDCL:~$ |
```

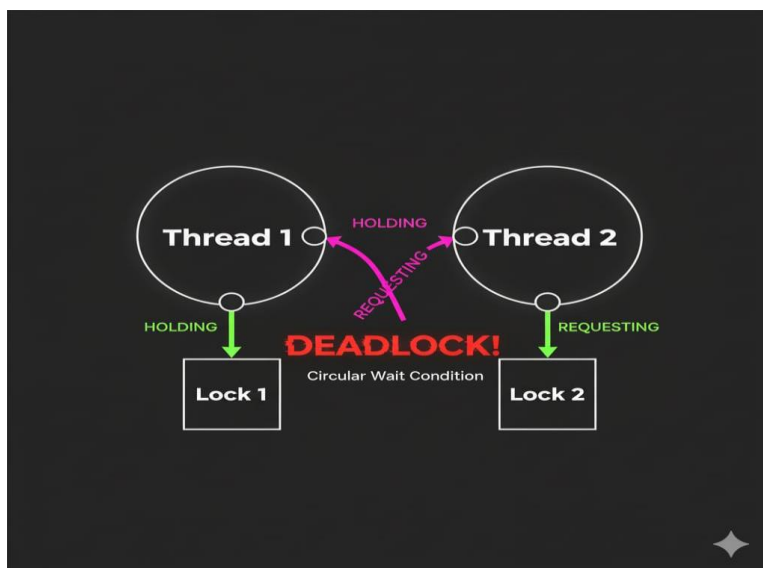
1. Program mengalami deadlock karena terjadi tunggu-menunggu melingkar (Circular Wait):
Thread 1 berhasil mendapatkan Lock 1 dan sedang menunggu Lock 2.

Thread 2 berhasil mendapatkan Lock 2 dan sedang menunggu Lock 1.

Karena Thread 1 tidak akan pernah melepaskan Lock 1 sebelum mendapatkan Lock 2, dan Thread 2 tidak akan pernah melepaskan Lock 2 sebelum mendapatkan Lock 1, maka kedua thread terjebak selamanya, menyebabkan program hang (deadlock).

2. Empat kondisi Coffman untuk terjadinya deadlock yang terpenuhi dalam program adalah:

- Mutual Exclusion (Saling Pengecualian)
- Hold and Wait (Pegang dan Tunggu)
- No Preemption (Tidak Dapat Direbut)
- Circular Wait (Menunggu Melingkar)



Tugas 2: Implementasi Pencegahan

Modifikasi program `deadlock_simple.c` menggunakan teknik:

1. Lock ordering
2. Try-lock dengan retry
3. Bandingkan kedua metode tersebut

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
// Deklarasi dua mutex
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
// Fungsi untuk Thread 1 (Urutan Lock: lock1 -> lock2)
void* thread1_func(void* arg) {
    printf("Thread 1: Mencoba mengambil Lock 1...\n");
    pthread_mutex_lock(&lock1);
    printf("Thread 1: Berhasil dapat Lock 1!\n");
    sleep(1);
    printf("Thread 1: Mencoba mengambil Lock 2...\n");
    pthread_mutex_lock(&lock2); // Lock kedua
    printf("Thread 1: Berhasil dapat Lock 2!\n");
    printf("Thread 1: Selesai bekerja\n");
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    return NULL;
}
// Fungsi untuk Thread 2 (Urutan Lock: lock1 -> lock2)
void* thread2_func(void* arg) {
    // *** MODIFIKASI: Membalik urutan agar sama dengan Thread 1 ***
    printf("Thread 2: Mencoba mengambil Lock 1...\n");
    pthread_mutex_lock(&lock1); // Lock pertama HARUS lock1
    printf("Thread 2: Berhasil dapat Lock 1!\n");
    sleep(1);
    printf("Thread 2: Mencoba mengambil Lock 2...\n");
    pthread_mutex_lock(&lock2); // Lock kedua HARUS lock2
    printf("Thread 2: Berhasil dapat Lock 2!\n");
    printf("Thread 2: Selesai bekerja\n");
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    return NULL;
}
int main() {
    pthread_t thread1, thread2;
    printf("=== MODIFIKASI DENGAN LOCK ORDERING ===\n\n");
    pthread_create(&thread1, NULL, thread1_func, NULL);
    pthread_create(&thread2, NULL, thread2_func, NULL);
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
    printf("\n=== PROGRAM SELESAI (Deadlock Terhindar) ===\n");
    return 0;
}
```

```
chandraLinux@LAPTOP-E5CDVDCL:~$ nano deadlock_simple.c
chandraLinux@LAPTOP-E5CDVDCL:~$ gcc deadlock_simple.c -o deadlock_simple -lpthread
chandraLinux@LAPTOP-E5CDVDCL:~$ ./deadlock_simple
=== MODIFIKASI DENGAN LOCK ORDERING ===

Thread 1: Mencoba mengambil Lock 1...
Thread 2: Mencoba mengambil Lock 1...
Thread 1: Berhasil dapat Lock 1!
Thread 1: Mencoba mengambil Lock 2...
Thread 1: Berhasil dapat Lock 2!
Thread 1: Selesai bekerja
Thread 2: Berhasil dapat Lock 1!
Thread 2: Mencoba mengambil Lock 2...
Thread 2: Berhasil dapat Lock 2!
Thread 2: Selesai bekerja

=== PROGRAM SELESAI (Deadlock Terhindar) ===
chandraLinux@LAPTOP-E5CDVDCL:~$ |
```

Fitur	1. Lock Ordering (Pengurutan Kunci)	2. Try-Lock dengan Retry (Coba Kunci dengan Ulangi)
Kategori	Pencegahan (Prevention)	Penghindaran (Avoidance)
Prinsip Utama	Menghilangkan kondisi Circular Wait .	Menghilangkan kondisi Hold and Wait .
Cara Kerja	Semua <i>thread</i> wajib mengambil <i>lock</i> dalam urutan yang telah ditetapkan (misalnya, selalu Lock 1 lalu Lock 2).	<i>Thread</i> mencoba mendapatkan <i>lock</i> kedua secara non-blocking. Jika gagal, ia melepaskan <i>lock</i> pertama dan mencoba lagi setelah jeda.
Keuntungan	Paling Efisien dan Sederhana. Tidak ada <i>overhead</i> atau <i>spinning</i> . <i>Deadlock</i> mustahil terjadi jika aturan urutan diikuti.	Fleksibel. Tidak perlu mengatur urutan global <i>lock</i> (berguna di sistem yang kompleks).
Kelemahan	Kurang Fleksibel. Sulit diterapkan pada sistem dengan banyak <i>lock</i> yang harus diakses secara kondisional atau dinamis.	Kurang Efisien. Menghasilkan lebih banyak <i>overhead</i> karena adanya retry (mengunci, melepaskan, menunggu, mengunci lagi). Risiko Livelock .
Kapan Dipilih	Untuk <i>resource</i> statis yang urutan aksesnya jelas.	Untuk <i>resource</i> yang dinamis atau ketika urutan global sulit/tidak mungkin dipertahankan.

♦

Tugas 3: Banker's Algorithm

Buat program baru dengan data:

- 4 proses
- 3 jenis resource
- Available: [5, 4, 3]
- Implementasikan request resource dan cek keamanan sistem

```
#include <stdio.h>
#include <stdbool.h>
#include <string.h>
// Definisi Jumlah Proses (P) dan Sumber Daya (R)
#define P 4
#define R 3
// --- DATA INISIAL ---
// Sumber daya yang tersedia saat ini (A, B, C)
int available[R] = {5, 4, 3};

// Jumlah maksimum sumber daya yang mungkin diminta setiap proses
// R1 R2 R3
int max[P][R] = {
    {3, 2, 2}, // P0
    {6, 1, 3}, // P1
    {3, 1, 4}, // P2
    {4, 2, 2}  // P3
};

// Jumlah sumber daya yang saat ini dialokasikan ke setiap proses
// R1 R2 R3
int allocation[P][R] = {
    {1, 0, 0}, // P0
    {2, 1, 1}, // P1
    {1, 1, 2}, // P2
    {0, 0, 1}  // P3
};

// Kebutuhan (Need) = Max - Allocation
int need[P][R];

// --- FUNGSI UTAMA ALGORITMA BANKER'S (SAFETY ALGORITHM) ---
bool is_safe(int temp_allocation[P][R], int temp_available[R]) {
    int work[R];
    bool finish[P] = {false};
    int safe_sequence[P];
    int count = 0;

    // 1. Inisialisasi Work = Available
    memcpy(work, temp_available, R * sizeof(int));
    // Hitung ulang Need untuk sementara
    for (int i = 0; i < P; i++) {
        for (int j = 0; j < R; j++) {
            need[i][j] = max[i][j] - temp_allocation[i][j];
        }
    }

    // 2. Cari Proses yang memenuhi kriteria: Need <= Work
    while (count < P) {
        bool found = false;
        for (int i = 0; i < P; i++) {
            if (finish[i] == false) {
                int j;
                for (j = 0; j < R; j++) {
                    if (need[i][j] > work[j])
                        break;
                }
                // Jika semua Need[i] <= Work terpenuhi
                if (j == R) {
                    // 3. Work = Work + Allocation[i]
                    for (int k = 0; k < R; k++) {
                        work[k] += temp_allocation[i][k];
                    }
                    safe_sequence[count++] = i;
                    finish[i] = true;
                    found = true;
                }
            }
        }
        // Jika tidak ada proses yang ditemukan dalam satu iterasi penuh
        if (found == false) {
            return false; // Sistem tidak aman
        }
    }

    // 4. Jika semua proses selesai
    printf("Sistem AMAN. Safe Sequence: <P%d", safe_sequence[0]);
    for (int i = 1; i < P; i++) {
        printf(", P%d", safe_sequence[i]);
    }
    printf("\n");
    return true; // Sistem aman
}
```

```
// --- FUNGSI REQUEST SUMBER DAYA (RESOURCE REQUEST ALGORITHM) ---
void request_resources(int process_id, int request[R]) {
    printf("\n--- Simulasi Permintaan Sumber Daya ---\n");
    printf("P%d meminta R: [%d, %d, %d]\n", process_id, request[0], request[1],
        request[2]);

    // Cek 1: Request <= Need
    for (int i = 0; i < R; i++) {
        if (request[i] > need[process_id][i]) {
            printf("Gagal! Permintaan melebihi kebutuhan maksimum (Need) P%d.\n",
                process_id);
            return;
        }
    }

    // Cek 2: Request <= Available
    for (int i = 0; i < R; i++) {
        if (request[i] > available[i]) {
            printf("Menunggu! Sumber daya tidak cukup saat ini (Request >
                Available).\n");
            return;
        }
    }

    // --- Jika kedua cek lolos, Coba Alokasikan sementara ---
    // Salin status Allocation dan Available saat ini
    int temp_allocation[R];
    int temp_available[R];
    for (int i = 0; i < R; i++) {
        for (int j = 0; j < P; j++) {
            temp_allocation[i][j] = allocation[i][j];
        }
    }

    memcpy(temp_available, available, R * sizeof(int));
    // Lakukan alokasi sementara (Trial Allocation)
    for (int i = 0; i < R; i++) {
        temp_available[i] -= request[i];
        temp_allocation[process_id][i] += request[i];
        // Need dihitung ulang di dalam is_safe()
    }

    // Cek 3: Cek Keamanan dengan alokasi sementara
    printf("Mencoba alokasi sementara dan mengecek keamanan sistem...\n");
    if (is_safe(temp_allocation, temp_available)) {
        printf("Permintaan DISAHKAN! Sistem tetap dalam keadaan aman.\n");
        // Update status sistem secara permanen
        memcpy(available, temp_available, R * sizeof(int));
        for (int i = 0; i < R; i++) {
            allocation[process_id][i] += request[i];
        }
    } else {
        printf("Permintaan DITOLAK! Alokasi akan menyebabkan sistem berada dalam
            keadaan TIDAK AMAN.\n");
    }
}

// --- PROGRAM UTAMA ---
```

```
int main() {
    printf("=== ALGORITMA BANKER'S SIMULATOR ===\n");
    printf("Initial Available: R: [%d, %d, %d]\n", available[0], available[1],
        available[2]);

    // --- Langkah 1: Cek Keamanan Kondisi Awal ---
    printf("\n--- Cek Keamanan Kondisi Awal ---\n");
    if (!is_safe(allocation, available)) {
        printf("Kondisi awal sistem TIDAK AMAN!\n");
    }

    // --- Langkah 2: Simulasi Permintaan P1 ---
    // Permintaan 1: P1 meminta [1, 0, 1]
    int req1[R] = {1, 0, 1};
    request_resources(1, req1); // P1
    // Tampilkan Available setelah permintaan
    printf("\nAvailable Baru: R: [%d, %d, %d]\n", available[0], available[1],
        available[2]);

    // --- Langkah 3: Simulasi Permintaan P3 yang menyebabkan Deadlock (Trial) ---
    // P3 membutuhkan [4, 2, 2] (Max) - [0, 0, 1] (Alloc) = [4, 2, 1] (Need)
    // Coba minta [1, 1, 0] yang cukup di Available (saat ini [4, 3, 2] jika req1
    // disahkan)
    int req2[R] = {1, 1, 0};
    request_resources(3, req2); // P3
    return 0;
}
```

```
chandr@linux@LAPTOP-E5CDVDCL:~$ nano bankers_sim.c
chandr@linux@LAPTOP-E5CDVDCL:~$ gcc bankers_sim.c -o bankers_sim
chandr@linux@LAPTOP-E5CDVDCL:~$ ./bankers_sim
=== ALGORITMA BANKER'S SIMULATOR ===
Initial Available: R: [5, 4, 3]

--- Cek Keamanan Kondisi Awal ---
Sistem AMAN. Safe Sequence: <P0, P1, P2, P3>

--- Simulasi Permintaan Sumber Daya ---
P1 meminta R: [1, 0, 1]
Mencoba alokasi sementara dan mengecek keamanan sistem...
Sistem AMAN. Safe Sequence: <P0, P1, P2, P3>
Permintaan DISAHKAN! Sistem tetap dalam keadaan aman.

Available Baru: R: [4, 4, 2]

--- Simulasi Permintaan Sumber Daya ---
P3 meminta R: [1, 1, 0]
Mencoba alokasi sementara dan mengecek keamanan sistem...
Sistem AMAN. Safe Sequence: <P0, P1, P2, P3>
Permintaan DISAHKAN! Sistem tetap dalam keadaan aman.
chandr@linux@LAPTOP-E5CDVDCL:~$
```

Tugas 4: Kasus Nyata (Real Case)

Buat simulasi deadlock untuk kasus: "Dua orang ingin transfer uang antar rekening secara bersamaan"

- Orang A: transfer dari Rekening 1 ke Rekening 2
- Orang B: transfer dari Rekening 2 ke Rekening 1
- Implementasikan deadlock prevention

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <stdlib.h> // Untuk rand()
// Saldo Awal dan Mutex (Resource)
long long saldo1 = 1000;
long long saldo2 = 1000;
pthread_mutex_t rek1_lock = PTHREAD_MUTEX_INITIALIZER; // ID 1
pthread_mutex_t rek2_lock = PTHREAD_MUTEX_INITIALIZER; // ID 2
// Fungsi Transfer A (Rek 1 -> Rek 2)
void* transfer_A(void* arg) {
    long long jumlah = 100;
    // --- LOCK ORDERING: Selalu kunci Rekening ID kecil dulu (Rek 1) ---
    pthread_mutex_lock(&rek1_lock);
    printf("Thread A: Lock Rekening 1 berhasil.\n");
    usleep(10000); // Simulasi kerja/jeda
    pthread_mutex_lock(&rek2_lock);
    printf("Thread A: Lock Rekening 2 berhasil.\n");
    // Transaksi
    if (saldo1 >= jumlah) {
        saldo1 -= jumlah;
        saldo2 += jumlah;
        printf("Thread A: Sukses! (R1: %lld, R2: %lld)\n", saldo1, saldo2);
    }
    pthread_mutex_unlock(&rek2_lock);
    pthread_mutex_unlock(&rek1_lock);
    return NULL;
}
// Fungsi Transfer B (Rek 2 -> Rek 1)
void* transfer_B(void* arg) {
    long long jumlah = 50;
    // --- LOCK ORDERING: Selalu kunci Rekening ID kecil dulu (Rek 1) ---
    pthread_mutex_lock(&rek1_lock); // Meskipun butuh Rek 2 dulu, harus kunci Rek 1
    (ID 1)
    printf("Thread B: Lock Rekening 1 berhasil.\n");
    usleep(10000); // Simulasi kerja/jeda
    pthread_mutex_lock(&rek2_lock);
    printf("Thread B: Lock Rekening 2 berhasil.\n");
    // Transaksi
    if (saldo2 >= jumlah) {
        saldo2 -= jumlah;
        saldo1 += jumlah;
        printf("Thread B: Sukses! (R1: %lld, R2: %lld)\n", saldo1, saldo2);
    }
    pthread_mutex_unlock(&rek2_lock);
    pthread_mutex_unlock(&rek1_lock);
    return NULL;
}
int main() {
    pthread_t threadA, threadB;
    printf("=== SIMULASI TRANSFER (LOCK ORDERING) ===\n");
    printf("Saldo Awal: R1=%lld, R2=%lld\n\n", saldo1, saldo2);
    pthread_create(&threadA, NULL, transfer_A, NULL);
    pthread_create(&threadB, NULL, transfer_B, NULL);
    pthread_join(threadA, NULL);
    pthread_join(threadB, NULL);
    printf("\n=== PROGRAM SELESAI ===\n");
    printf("Saldo Akhir: R1=%lld, R2=%lld (Total: %lld)\n", saldo1, saldo2, saldo1 +
saldo2);
    return 0;
}
```

```
chandra@linux@LAPTOP-E5CDVDCL:~$ nano deadlock_prevention.c
chandra@linux@LAPTOP-E5CDVDCL:~$ gcc deadlock_prevention.c -o transfer_sim -pthread
chandra@linux@LAPTOP-E5CDVDCL:~$ ./transfer_sim
=== SIMULASI TRANSFER (LOCK ORDERING) ===
Saldo Awal: R1=1000, R2=1000

Thread A: Lock Rekening 1 berhasil.
Thread A: Lock Rekening 2 berhasil.
Thread A: Sukses! (R1: 900, R2: 1100)
Thread B: Lock Rekening 1 berhasil.
Thread B: Lock Rekening 2 berhasil.
Thread B: Sukses! (R1: 950, R2: 1050)

=== PROGRAM SELESAI ===
Saldo Akhir: R1=950, R2=1050 (Total: 2000)
chandra@linux@LAPTOP-E5CDVDCL:~$ |
```